5 Linux 物理内存分配全链路实现 2
5.1 内核物理内存分配接口 2
5.2 物理内存分配掩码 gfp_mask 4
5.3 物理内存分配相关函数源码分析 8
5.3.1 影响内核分配内存行为的标识 9
5.3.2 内存分配__alloc_pages 函数分析 11
5.3.2.1 prepare_alloc_pages 函数分析 15
5.3.2.2 慢速分配入口函数 alloc_pages_slowpath 函数分析 16
5.3.2.3 慢速分配函数 alloc_pages_slowpath 函数中 nopage 模块分析 26

5 Linux 物理内存分配全链路实现

5.1 内核物理内存分配接口

内核中用于物理内存分配的几个核心接口：

内存分配接口：

```
struct page *alloc_pages(gfp_t gfp, unsigned int order);

#define alloc_page(gfp_mask) alloc_pages(gfp_mask, 0)

unsigned long get_zeroed_page(gfp_t gfp_mask)

#define __get_dma_pages(gfp_mask, order) \
        __get_free_pages((gfp_mask) | GFP_DMA, (order))

unsigned long __get_free_pages(gfp_t gfp_mask, unsigned int order)

#define __get_free_page(gfp_mask) \
        __get_free_pages((gfp_mask), 0)
```

内存释放接口：

```
void __free_pages(struct page *page, unsigned int order)

void free_pages(unsigned long addr, unsigned int order)

#define __free_page(page) __free_pages((page), 0)

#define free_page(addr) free_pages((addr), 0)
```

这几个物理内存分配接口全部是基于伙伴系统的，伙伴系统有一个特点就是它所分配的物理内存页全部都是物理上连续的，并且只能分配 2 的整数幂个页，这里的整数幂在内核中称之为分配阶。

分配连续内存页的接口：

alloc_pages 用于向底层伙伴系统申请 2 的 order 次幂个连续物理内存页组成的内存块，参数中的 unsigned int order 表示向底层伙伴系统指定的分配阶，参数 gfp_t gfp 是内核中定义的一个用于规范物理内存分配行为的修饰符。

alloc_page 宏用于**单内存页**分配的场景，其底层还是依赖了 alloc_pages 函数，只不过 order 指定为 0。

当系统中空闲的物理内存无法满足内存分配时，就会导致内存分配失败，alloc_pages，alloc_page 就会返回空指针 NULL 。vmalloc 分配机制底层就是用的 alloc_page。

在物理内存分配成功的情况下， alloc_pages，alloc_page 函数返回的都是指向其申请的物理内存块第一个物理内存页的指针。

可以理解成返回的是一块物理内存，而 CPU 可以直接访问的却是虚拟内存，所以内核又提供了一个函数 __get_free_pages ，该函数直接返回物理内存页的虚拟内存地址。用户

可以直接使用。

__get_free_pages 函数在使用方式上和 alloc_pages 是一样的，函数参数的含义也是一样，只不过一个是返回物理内存页的虚拟内存地址，一个是直接返回物理内存页。

__get_free_pages 函数的底层也是基于 alloc_pages 实现的，只不过多了一层虚拟地址转换的工作。

```
unsigned long __get_free_pages(gfp_t gfp_mask, unsigned int order)
{
    struct page *page;
    // 不能在高端内存中分配物理页，因为无法直接映射获取虚拟内存地址
    page = alloc_pages(gfp_mask & ~__GFP_HIGHMEM, order);
    if (!page)
        return 0;
     // 将直接映射区中的物理内存页转换为虚拟内存地址
    return (unsigned long) page_address(page);
}
```

page_address 函数用于将给定的物理内存页 page 转换为它的虚拟内存地址，不过这里只适用于内核虚拟内存空间中的直接映射区，因为在直接映射区中虚拟内存地址到物理内存地址是直接映射的，虚拟内存地址减去一个固定的偏移就可以直接得到物理内存地址。

如果物理内存页处于高端内存中，则不能这样直接进行转换，在通过 alloc_pages 函数获取物理内存页 page 之后，需要调用 kmap 映射将 page 映射到内核虚拟地址空间中。

内核也提供了 **__get_free_page** 用于只分配单个物理内存页的场景，底层还是依赖于 __get_free_pages 函数，参数 order 指定为 0 。

无论是 alloc_pages 也好还是 __get_free_pages 也好，它们申请到的内存页中包含的数据在一开始都不是空白的，而是内核随机产生的一些垃圾信息，但其实这些信息可能并不都是完全随机的，很有可能随机的包含一些敏感的信息。

从使用安全的角度考虑，内核又提供了一个函数 **get_zeroed_page**，将从伙伴系统中申请到内存页全部初始化填充为 0 ，这在分配物理内存页给用户空间使用的时候非常有用。

```
unsigned long get_zeroed_page(gfp_t gfp_mask)
{
    return __get_free_page(gfp_mask | __GFP_ZERO);
}
```

get_zeroed_page 函数底层也依赖于 __get_free_pages，指定的分配阶 order 也是 0，表示从伙伴系统中只申请一个物理内存页并初始化填充 0 。

__get_dma_pages 函数，专门用于从 DMA 内存区域分配适用于 DMA 的物理内存页。其底层也是依赖于 __get_free_pages 函数。

```
#define __get_dma_pages(gfp_mask, order) \
        __get_free_pages((gfp_mask) | GFP_DMA, (order))
```

这些底层依赖于 __get_free_pages 的物理内存分配函数，在遇到内存分配失败的情况下都会返回 0 。

内存释放接口：

__free_pages ： 同 alloc_pages 函数对应，用于释放一个或者 2 的 order 次幂个内存页，释放的物理内存区域起始地址由该区域中的第一个 page 实例指针表示，也就是参数里的 struct page *page 指针。

free_pages：同 __get_free_pages 函数对应，与 __free_pages 函数的区别是在释放物

理内存时，使用了虚拟内存地址而不是 page 指针。

另外内核也提供了 **__free_page** 和 **free_page** 两个宏，专门用于释放单个物理内存页。

5.2 物理内存分配掩码 gfp_mask

调用物理内存分配接口时，分配掩码 gfp_mask 指定了从哪些内核物理内存区域中分配内存页。前缀 gfp 是 get free page 的缩写，意思是在获取空闲物理内存页的时候需要指定的分配掩码 gfp_mask。gfp_mask 中的低 4 位用来表示应该从哪个物理内存区域 zone 中获取内存页 page。

gfp_mask 掩码中这些区域修饰符 zone modifiers 定义在内核 **include\linux\gfp_types.h** 中，

```
#define ___GFP_DMA      0x01u
#define ___GFP_HIGHMEM      0x02u
#define ___GFP_DMA32        0x04u
#define ___GFP_MOVABLE      0x08u
```

内核对物理内存的分配主要是落在 ZONE_NORMAL 区域中，如果我们不指定物理内存的分配区域，那么内核会默认从 ZONE_NORMAL 区域中分配内存，如果 ZONE_NORMAL 区域中的空闲内存不够，内核则会降级到 ZONE_DMA 区域中分配。因此，内核中没有定义 ___GFP_NORMAL 的掩码。

文件 include\linux\gfp.h 中定义了一个叫做 gfp_zone 的函数，这个函数用于将我们在物理内存分配接口中指定的 gfp_mask 掩码转换为物理内存区域，返回的这个物理内存区域是内存分配的最高级内存区域，如果这个最高级内存区域不足以满足内存分配的需求，则按照 ZONE_HIGHMEM -> ZONE_NORMAL -> ZONE_DMA 的顺序依次降级。

```
static inline enum zone_type gfp_zone(gfp_t flags)
{
    enum zone_type z;
    int bit = (__force int) (flags & GFP_ZONEMASK);

    z = (GFP_ZONE_TABLE >> (bit * GFP_ZONES_SHIFT)) &
                    ((1 << GFP_ZONES_SHIFT) - 1);
    VM_BUG_ON((GFP_ZONE_BAD >> bit) & 1);
    return z;
}
```

对比低版本 2.6.24 中的实现，在这一版中可以更加清晰的看到 gfp_zone 函数原本的面貌。

```
static inline enum zone_type gfp_zone(gfp_t flags)
{
 int base = 0;

#ifdef CONFIG_NUMA
 if (flags & __GFP_THISNODE)
  base = MAX_NR_ZONES;
#endif
```

```
#ifdef CONFIG_ZONE_DMA
 if (flags & __GFP_DMA)
  return base + ZONE_DMA;
#endif
#ifdef CONFIG_ZONE_DMA32
 if (flags & __GFP_DMA32)
  return base + ZONE_DMA32;
#endif
 if ((flags & (__GFP_HIGHMEM | __GFP_MOVABLE)) ==
   (__GFP_HIGHMEM | __GFP_MOVABLE))
  return base + ZONE_MOVABLE;
#ifdef CONFIG_HIGHMEM
 if (flags & __GFP_HIGHMEM)
  return base + ZONE_HIGHMEM;
#endif
    // 默认从 normal 区域中分配内存
 return base + ZONE_NORMAL;
}
}
```

核心逻辑主要如下：

1. 只要掩码 flags 中设置了 __GFP_DMA，则不管 __GFP_HIGHMEM 有没有设置，内存分配都只会在 ZONE_DMA 区域中分配。

2. 如果掩码只设置了 ZONE_HIGHMEM，则在物理内存分配时，优先在 ZONE_HIGHMEM 区域中进行分配，如果容量则降级到 ZONE_NORMAL 中(后续流程触发)，如果还是不够则进一步降级至 ZONE_DMA 中分配。

3. 如果掩码既没有设置 ZONE_HIGHMEM 也没有设置 __GFP_DMA，则走到最后的分支，默认优先从 ZONE_NORMAL 区域中进行内存分配，如果容量不够则降级至 ZONE_DMA 区域中分配。

4. 单独设置 __GFP_MOVABLE 其实并不会影响内核的分配策略，我们如果想要让内核在 ZONE_MOVABLE 区域中分配内存需要同时指定 __GFP_MOVABLE 和 __GFP_HIGHMEM。

ZONE_MOVABLE 只是内核定义的一个虚拟内存区域，目的是避免内存碎片和支持内存热插拔。上述介绍的 ZONE_HIGHMEM，ZONE_NORMAL，ZONE_DMA 才是真正的物理内存区域，ZONE_MOVABLE 虚拟内存区域中的物理内存来自于上述三个物理内存区域。

在 32 位系统中 ZONE_MOVABLE 虚拟内存区域中的物理内存页来自于 ZONE_HIGHMEM。在 64 位系统中 ZONE_MOVABLE 虚拟内存区域中的物理内存页来自于 ZONE_NORMAL 或者 ZONE_DMA 区域。

gfp_t 掩码设置方式与其对应的内存区域降级策略：

gfp_t 掩码	内存区域降级策略
什么都没有设置	ZONE_NORMAL -> ZONE_DMA
__GFP_DMA	ZONE_DMA
__GFP_DMA & __GFP_HIGHMEM	ZONE_DMA
__GFP_HIGHMEM	ZONE_HIGHMEM -> ZONE_NORMAL -> ZONE_DMA

除了上述介绍 gfp_t 掩码中的这四个物理内存区域修饰符之外，内核还定义了一些规范内存分配行为的修饰符，这些行为修饰符并不会限制内核从哪个物理内存区域中分配内存，而是会限制物理内存分配的行为，这些内存分配行为修饰符同样也是定义在 /include/linux/gfp.h 文件中：

```
#define ___GFP_RECLAIMABLE  0x10u
#define ___GFP_HIGH     0x20u
#define ___GFP_IO       0x40u
#define ___GFP_FS       0x80u
#define ___GFP_ZERO     0x100u
/* 0x200u unused */
#define ___GFP_DIRECT_RECLAIM   0x400u
#define ___GFP_KSWAPD_RECLAIM   0x800u
#define ___GFP_WRITE        0x1000u
#define ___GFP_NOWARN       0x2000u
#define ___GFP_RETRY_MAYFAIL    0x4000u
#define ___GFP_NOFAIL       0x8000u
#define ___GFP_NORETRY      0x10000u
#define ___GFP_MEMALLOC     0x20000u
#define ___GFP_COMP     0x40000u
#define ___GFP_NOMEMALLOC   0x80000u
```

___GFP_RECLAIMABLE 用于指定分配的页面是可以回收的，

___GFP_MOVABLE 则是用于指定分配的页面是可以移动的，这两个标志会影响底层的伙伴系统从哪个区域中去获取空闲内存页

___GFP_HIGH 表示该内存分配请求是高优先级的，内核急切的需要内存，如果内存分配失败则会给系统带来非常严重的后果，设置该标志通常内存是不允许分配失败的，如果空闲内存不足，则会从紧急预留内存中分配。

___GFP_IO 表示内核在分配物理内存的时候可以发起磁盘 IO 操作。比如当内核在进行内存分配的时候，发现物理内存不足，这时需要将不经常使用的内存页置换到 SWAP 分区或者 SWAP 文件中，这时就涉及到了 IO 操作，如果设置了该标志，表示允许内核将不常用的内存页置换出去。

___GFP_FS 允许内核执行底层文件系统操作，在与 VFS 虚拟文件系统层相关联的内核子系统中必须禁用该标志，否则可能会引起文件系统操作的循环递归调用，因为在设置 ___GFP_FS 标志分配内存的情况下，可能会引起更多的文件系统操作，而这些文件系统的操作可能又会进一步产生内存分配行为，这样一直递归持续下去。

___GFP_ZERO 在内核分配内存成功之后，将内存页初始化填充字节 0 。

___GFP_ATOMIC 该标志的设置表示内存在分配物理内存的时候不允许睡眠必须是原子性地进行内存分配。比如在中断处理程序中，就不能睡眠，因为中断程序不能被重新调度。同时也不能在持有自旋锁的进程上下文中睡眠，因为可能导致死锁。综上所述这个标志只能用在不能被重新安全调度的进程上下文中。

___GFP_DIRECT_RECLAIM 表示内核在进行内存分配的时候，可以进行直接内存回收。当剩余内存容量低于水位线 _watermark[WMARK_MIN] 时，说明此时的内存容量已经非常危险了，如果进程在这时请求内存分配，内核就会进行直接内存回收，直到内存水位线恢复到 _watermark[WMARK_HIGH] 之上。

___GFP_KSWAPD_RECLAIM 表示内核在分配内存的时候，如果剩余内存容量在 _watermark[WMARK_MIN] 与 _watermark[WMARK_LOW] 之间时，内核就会唤醒 kswapd 进程开始异步内存回收，直到剩余内存高于 _watermark[WMARK_HIGH] 为止。

___GFP_NOWARN 表示当内核分配内存失败时，抑制内核的分配失败错误报告。

___GFP_RETRY_MAYFAIL 在内核分配内存失败的时候，允许重试，但重试仍然可能失败，重试若干次后停止。

___GFP_NORETRY 标志表示分配内存失败时不允许重试。

___GFP_NOFAIL 在内核分配失败时一直重试直到成功为止。

___GFP_HARDWALL 该标志限制了内核分配内存的行为只能在当前进程分配到的 CPU 所关联的 NUMA 节点上进行分配，当进程可以运行的 CPU 受到限时，该标志才会有意义，如果进程允许在所有 CPU 上运行则该标志没有意义。

___GFP_THISNODE 该标志限制了内核分配内存的行为只能在当前 NUMA 节点或者在指定 NUMA 节点中分配内存，如果内存分配失败不允许从其他备用 NUMA 节点中分配内存。

___GFP_MEMALLOC 允许内核在分配内存时可以从所有内存区域中获取内存，包括从紧急预留内存中获取。但使用该标示时需要保证进程在获得内存之后会很快的释放掉内存不会过长时间的占用，尤其要警惕避免过多的消耗紧急预留内存区域中的内存。

___GFP_NOMEMALLOC 标志用于明确禁止内核从紧急预留内存中获取内存。___GFP_NOMEMALLOC 标识的优先级要高于 ___GFP_MEMALLOC。

内核中各种标准情形下用到的 gfp_t 掩码组合：

```
#define GFP_ATOMIC  (__GFP_HIGH|__GFP_ATOMIC|__GFP_KSWAPD_RECLAIM)
#define GFP_KERNEL  (__GFP_RECLAIM | __GFP_IO | __GFP_FS)
#define GFP_KERNEL_ACCOUNT (GFP_KERNEL | __GFP_ACCOUNT)
#define GFP_NOWAIT  (__GFP_KSWAPD_RECLAIM | __GFP_NOWARN)
#define GFP_NOIO    (__GFP_RECLAIM)
#define GFP_NOFS    (__GFP_RECLAIM | __GFP_IO)
#define GFP_USER    (__GFP_RECLAIM | __GFP_IO | __GFP_FS | __GFP_HARDWALL)
#define GFP_DMA     __GFP_DMA
#define GFP_DMA32   __GFP_DMA32
#define GFP_HIGHUSER    (GFP_USER | __GFP_HIGHMEM)
```

GFP_ATOMIC 是掩码 __GFP_HIGH，__GFP_ATOMIC，__GFP_KSWAPD_RECLAIM 的组合，表示内存分配行为必须是原子的，是高优先级的。在任何情况下都不允许睡眠，如果空闲内存不够，则会从紧急预留内存中分配。该标志适用于中断程序，以及持有自旋锁的进程上下文中。

GFP_KERNEL 是内核中最常用的标志，该标志设置之后内核的分配内存行为可能会阻塞睡眠，可以允许内核置换出一些不活跃的内存页到磁盘中。适用于可以重新安全调度的进程上下文中。

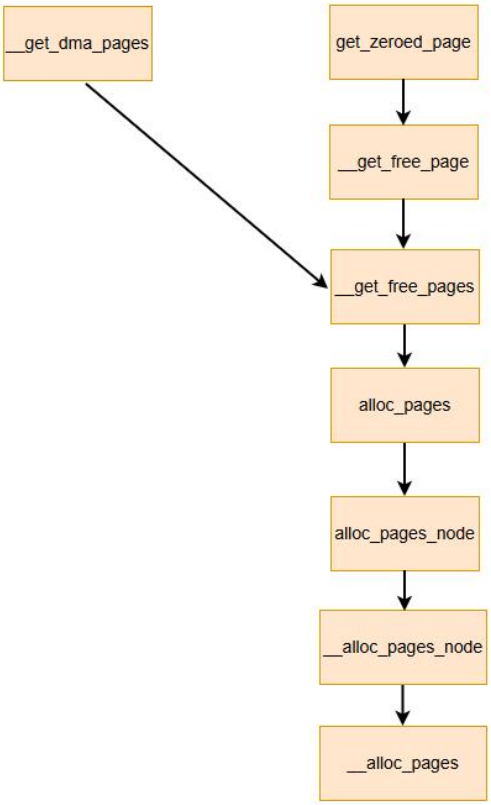
GFP_NOIO 和 **GFP_NOFS** 分别禁止内核在分配内存时进行磁盘 IO 和 文件系统 IO 操作。

GFP_USER 用于映射到用户空间的内存分配，通常这些内存可以被内核或者硬件直接访问，比如硬件设备会将 Buffer 直接映射到用户空间中。

GFP_HIGHUSER 用于给用户空间分配高端内存，因为在用户虚拟内存空间中，都是通过页表来访问非直接映射的高端内存区域，所以用户空间一般使用的是高端内存区域 ZONE_HIGHMEM

5.3 物理内存分配相关函数源码分析

内存分配接口源码流程图：



内存分配的任务最终会落在 alloc_pages 这个接口函数中，在 alloc_pages 中会调用 alloc_pages_node 进而调用 __alloc_pages_node 函数，最终通过 __alloc_pages 函数正式进入内核内存分配。__alloc_pages 函数为 Linux 内核内存分配的核心入口函数

```
    static inline struct page *alloc_pages(gfp_t gfp_mask, unsigned int order)
{
    return alloc_pages_node(numa_node_id(), gfp_mask, order);
}
```

```
static inline struct page *alloc_pages_node(int nid, gfp_t gfp_mask,
                        unsigned int order)
{
    if (nid == NUMA_NO_NODE)
        nid = numa_mem_id();

    return __alloc_pages_node(nid, gfp_mask, order);
}
```

```
static inline struct page *
__alloc_pages_node(int nid, gfp_t gfp_mask, unsigned int order)
{
     // 校验指定的 NUMA 节点 ID 是否合法，不要越界
    VM_BUG_ON(nid < 0 || nid >= MAX_NUMNODES);
     // 指定节点必须是有效在线的
    warn_if_node_offline(nid, gfp_mask);

    return __alloc_pages(gfp_mask, order, nid, NULL);
}
```

numa_node_id() 获取运行当前进程的 CPU 所在的 NUMA 节点。

warn_if_node_offline(nid, gfp_mask) 确保指定的 NUMA 节点是有效在线的。

5.3.1 影响内核分配内存行为的标识

标识定义在内核文件 mm\internal.h 中：

```
#define ALLOC_WMARK_MIN     WMARK_MIN
#define ALLOC_WMARK_LOW     WMARK_LOW
#define ALLOC_WMARK_HIGH    WMARK_HIGH
#define ALLOC_NO_WATERMARKS 0x04 /* don't check watermarks at all */

/* Mask to get the watermark bits */
#define ALLOC_WMARK_MASK    (ALLOC_NO_WATERMARKS-1)
#define ALLOC_NON_BLOCK      0x10 /* Caller cannot block. Allow access
                      * to 25% of the min watermark or
                      * 62.5% if __GFP_HIGH is set.
                      */
#define ALLOC_MIN_RESERVE    0x20 /* __GFP_HIGH set. Allow access to 50%
                      * of the min watermark.
                      */
#define ALLOC_CPUSET         0x40 /* check for correct cpuset */
#define ALLOC_CMA        0x80 /* allow allocations from CMA areas */
#ifdef CONFIG_ZONE_DMA32
```

```
#define ALLOC_NOFRAGMENT    0x100 /* avoid mixing pageblock types */
#else
#define ALLOC_NOFRAGMENT      0x0
#endif
#define ALLOC_HIGHATOMIC    0x200 /* Allows access to MIGRATE_HIGHATOMIC */
#define ALLOC_KSWAPD        0x800 /* allow waking of kswapd,
__GFP_KSWAPD_RECLAIM set */
```

ALLOC_WMARK_MIN、ALLOC_WMARK_LOW、ALLOC_WMARK_HIGH、ALLOC_NO_WATERMARKS 这个四个内存水位线标识表示内核在分配内存时必须考虑内存的水位线，在不同的水位线下内存的分配行为也会有所不同。

内核会为 NUMA 节点中的每个物理内存区域 zone 定制三条用于指示内存容量的水位线，它们分别是：WMARK_MIN（页最小阈值），WMARK_LOW（页低阈值），WMARK_HIGH（页高阈值）。

这三个水位线定义在 include\linux\mmzone.h 文件中：

```
    enum zone_watermarks {
    WMARK_MIN,
    WMARK_LOW,
    WMARK_HIGH,
    WMARK_PROMO,
    NR_WMARK
};
```

三条水位线对应的 watermark 具体数值存储在每个物理内存区域 struct zone 结构中的 _watermark[NR_WMARK] 数组中。

```
struct zone {
    ......
    /* zone watermarks, access with *_wmark_pages(zone) macros */
    unsigned long _watermark[NR_WMARK];
    ......
    }
```

物理内存区域中不同水位线的含义以及内存分配在不同水位线下的行为如下图所示：

1. 当该物理内存区域的剩余内存容量高于 _watermark[WMARK_HIGH] 时，说明此时该物理内存区域中的内存容量非常充足，内存分配完全没有压力。

2. 当剩余内存容量在 _watermark[WMARK_LOW] 与_watermark[WMARK_HIGH] 之间时，说明此时内存有一定的消耗但是还可以接受，能够继续满足进程的内存分配需求。

3. 当剩余内存容量在 _watermark[WMARK_MIN] 与 _watermark[WMARK_LOW] 之间时，说明此时内存容量已经有点危险了，内存分配面临一定的压力，但是还可以满足进程此时的内存分配要求，当给进程分配完内存之后，就会唤醒 kswapd 进程开始内存回收，直到剩余内存高于 _watermark[WMARK_HIGH] 为止。在这种情况下，进程的内存分配会触发内存回收，但请求进程本身不会被阻塞，由内核的 kswapd 进程异步回收内存。

4. 当剩余内存容量低于 _watermark[WMARK_MIN] 时，说明此时的内存容量已经非常危险了，如果进程在这时请求内存分配，内核就会进行直接内存回收，这时内存回收的任务将会由请求进程同步完成。

物理内存区域 zone 的剩余内存是需要刨去 lowmem_reserve 预留内存大小（用于紧急内存分配）。也就是说 zone 里被伙伴系统所管理的内存并不包含 lowmem_reserve 预留内存。

ALLOC_NO_WATERMARKS 表示在内存分配过程中完全不会考虑上述三个水位线的影响。

ALLOC_WMARK_HIGH 表示在内存分配的时候，当前物理内存区域 zone 中剩余内存页的数量至少要达到 _watermark[WMARK_HIGH] 水位线，才能进行内存的分配。

ALLOC_WMARK_LOW 和 **ALLOC_WMARK_MIN** 表示当前物理内存区域 zone 中剩余内存页的数量至少要达到水位线 _watermark[WMARK_LOW] 或者 _watermark[WMARK_MIN]，才能进行内存的分配。

ALLOC_HARDER 表示在内存分配的时候，会放宽内存分配规则的限制，所谓的放宽规则就是降低 _watermark[WMARK_MIN] 水位线，努力使内存分配最大可能成功。

在 gfp_t 掩码中设置了 ___GFP_HIGH 时，ALLOC_HIGH 标识才起作用，该标识表示当前内存分配请求是高优先级的，内核急切的需要内存，如果内存分配失败则会给系统带来非常严重的后果，设置该标志通常内存是不允许分配失败的，如果空闲内存不足，则会从紧急预留内存中分配。

ALLOC_CPUSET 表示内存只能在当前进程所允许运行的 CPU 所关联的 NUMA 节点中进行分配。比如使用 cgroup 限制进程只能在某些特定的 CPU 上运行，那么进程所发起的内存分配请求，只能在这些特定 CPU 所在的 NUMA 节点中进行。

ALLOC_KSWAPD 表示允许唤醒 NUMA 节点中的 KSWAPD 进程，异步进行内存回收。

内核会为每个 NUMA 节点分配一个 kswapd 进程用于回收不经常使用的页面。

```
typedef struct pglist_data {
	......
	struct task_struct *kswapd; /* Protected by kswapd_lock */
	......
}
```

5.3.2 内存分配__alloc_pages 函数分析

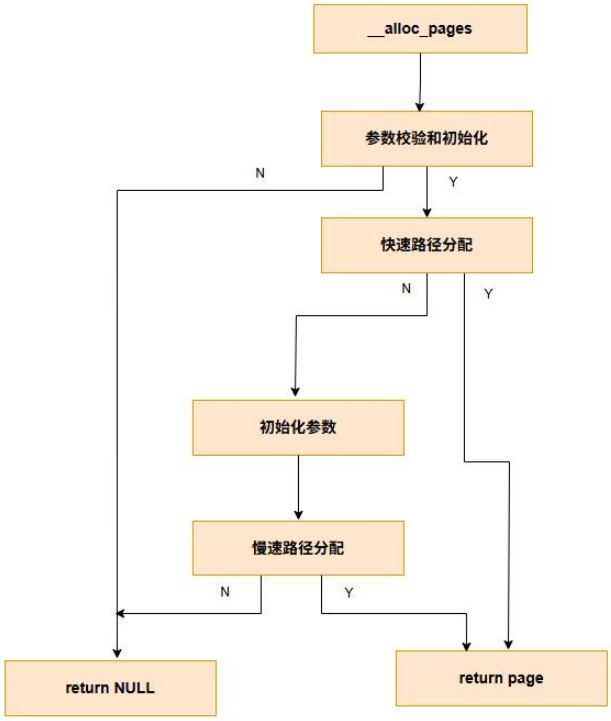
__alloc_pages 是内核内存分配的核心入口函数，整个内存分配的完整过程全部封装在里面。

```
/*
```

```
/*
 * This is the 'heart' of the zoned buddy allocator.
 */
struct page *__alloc_pages(gfp_t gfp, unsigned int order, int preferred_nid,
                           nodemask_t *nodemask)
{
    // 用于指向分配成功的内存
    struct page *page;
    // 内存区域中的剩余内存需要在 WMARK_LOW 水位线之上才能进行内存分配，否则失
败（初次尝试快速内存分配）
    unsigned int alloc_flags = ALLOC_WMARK_LOW;
    //内存分配掩码集合
    gfp_t alloc_gfp; /* The gfp_t that was actually used for allocation */
   //用于在不同内存分配辅助函数中传递参数
    struct alloc_context ac = { };
    // 检查用于向伙伴系统申请内存容量的分配阶 order 的合法性
    // 内核定义最大分配阶 MAX_ORDER -1 = 10，也就是说一次最多只能从伙伴系统中
申请 1024 个内存页。
    if (WARN_ON_ONCE_GFP(order > MAX_PAGE_ORDER, gfp))
        return NULL;
    //表示在内存分配期间进程可以休眠阻塞
    gfp &= gfp_allowed_mask;
    gfp = current_gfp_context(gfp);
    alloc_gfp = gfp;
    // 初始化 alloc_context，并为接下来的快速内存分配设置相关 gfp
    if (!prepare_alloc_pages(gfp, order, preferred_nid, nodemask, &ac,
            &alloc_gfp, &alloc_flags))
        // 提前判断本次内存分配是否能够成功，如果不能则尽早失败
        return NULL;
    // 避免内存碎片化的相关分配标识设置
    alloc_flags |= alloc_flags_nofragment(ac.preferred_zoneref->zone,
gfp);
    /* First allocation attempt */
    // 内存分配快速路径：第一次尝试从底层伙伴系统分配内存，注意此时是在
WMARK_LOW 水位线之上分配内存
    page = get_page_from_freelist(alloc_gfp, order, alloc_flags, &ac);
    if (likely(page))
    // 如果内存分配成功则直接返回
        goto out;
    // 流程走到这里表示内存分配在快速路径下失败
    // 这里需要恢复最初的内存分配标识设置，后续会尝试更加激进的内存分配策略
    alloc_gfp = gfp;
    ac.spread_dirty_pages = false;
    // 恢复最初的 node mask 因为它可能在第一次内存分配的过程中被改变
```

```
    // 本函数中 nodemask 起初被设置为 null
    ac.nodemask = nodemask;
    // 在第一次快速内存分配失败之后，说明内存已经不足了，内核需要做更多的工作
    // 比如通过 kswap 回收内存，或者直接内存回收等方式获取更多的空闲内存以满足
内存分配的需求
    // 所以下面的过程称之为慢速分配路径
    page = __alloc_pages_slowpath(alloc_gfp, order, &ac);
out:
    if (memcg_kmem_online() && (gfp & __GFP_ACCOUNT) && page &&
        unlikely(__memcg_kmem_charge_page(page, gfp, order) != 0)) {
        __free_pages(page, order);
        page = NULL;
    }
    trace_mm_page_alloc(page, order, alloc_gfp, ac.migratetype);
    kmsan_alloc_page(page, order, alloc_gfp);
    // 内存分配成功，直接返回 page。否则返回 NULL
    return page;
}
```

__alloc_pages 函数整体逻辑流程图：



其内存分配整体逻辑如下：

1. 首先内核会尝试在内存水位线 WMARK_LOW 之上快速的进行一次内存分配。这一点可以从开始的 unsigned int alloc_flags = ALLOC_WMARK_LOW 中看来。

2. 校验本次内存分配指定伙伴系统的分配阶 order 的有效性，伙伴系统在内核中的最大分配阶定义在 include\linux\mmzone.h 文件中，最大分配阶 MAX_ORDER -1 = 10，也就是一次最多只能从伙伴系统中申请 1024 个内存页，对应 4M 大小的连续物理内存。

3. 调用 prepare_alloc_pages 初始化 alloc_context ，用于在不同内存分配辅助函数中传递内存分配参数。为接下来即将进行的快速内存分配做准备。

```
struct alloc_context {
    //运行进程 CPU 所在 NUMA 节点以及其所有备用 NUMA 节点中允许内存分配的内存区域
    struct zonelist *zonelist;
     // NUMA  节点状态掩码
    nodemask_t *nodemask;
    // 内存分配优先级最高的内存区域 zone
    struct zoneref *preferred_zoneref;
    // 物理内存页的迁移类型分为：不可迁移，可回收，可迁移类型，防止内存碎片
    int migratetype;
    // 内存分配最高优先级的内存区域 zone
    enum zone_type highest_zoneidx;
     // 是否允许当前 NUMA 节点中的脏页均衡扩散迁移至其他 NUMA 节点
    bool spread_dirty_pages;
};
```

4. 调用 get_page_from_freelist 方法首次尝试在伙伴系统中进行内存分配，这次内存分配比较快速，只是快速的扫描一下各个内存区域中是否有足够的空闲内存能够满足本次内存分配，如果有则立马从伙伴系统中申请，如果没有立即返回， page 设置为 null，进行后续慢速内存分配处理。此时，快速内存分配是在 WMARK_LOW 水位线之上进行的。

5. 当快速内存分配失败之后，情况就会变得非常复杂，内核将不得不做更多的工作，比如开启 kswapd 进程异步内存回收，更极端的情况则需要进行直接内存回收，或者直接内存整理以获取更多的空闲连续内存。这一切的复杂逻辑全部封装在 __alloc_pages_slowpath 函数中。

__alloc_pages 函数内存分配逻辑以及与对应的内存水位线之间的关系如下图所示：

5.3.2.1 prepare_alloc_pages 函数分析

prepare_alloc_pages 初始化 alloc_context ，用于在不同内存分配辅助函数中传递内存分配参数，为接下来即将进行的快速内存分配做准备。

```
static inline bool prepare_alloc_pages(gfp_t gfp_mask, unsigned int order,
        int preferred_nid, nodemask_t *nodemask,
        struct alloc_context *ac, gfp_t *alloc_gfp,
        unsigned int *alloc_flags)
{
     // 根据 gfp_mask 掩码中的内存区域修饰符获取内存分配最高优先级的内存区域
zone
    ac->highest_zoneidx = gfp_zone(gfp_mask);
     // 从 NUMA 节点的备用节点链表中一次性获取允许进行内存分配的所有内存区域
    ac->zonelist = node_zonelist(preferred_nid, gfp_mask);
    ac->nodemask = nodemask;
    // 从 gfp_mask 掩码中获取页面迁移属性，
    //迁移属性分为：不可迁移，可回收，可迁移。
    ac->migratetype = gfp_migratetype(gfp_mask);
   // 如果使用 cgroup 将进程绑定限制在了某些 CPU 上，那么内存分配只能在
   // 这些绑定的 CPU 相关联的 NUMA 节点中进行
    if (cpusets_enabled()) {
        *alloc_gfp |= __GFP_HARDWALL;
        if (in_task() && !ac->nodemask)
            ac->nodemask = &cpuset_current_mems_allowed;
        else
            *alloc_flags |= ALLOC_CPUSET;
    }
    // 如果设置了允许直接内存回收，
    //那么内存分配进程则可能会导致休眠被重新调度
    might_alloc(gfp_mask);
    // 提前判断本次内存分配是否能够成功，如果不能则尽早失败
    if (should_fail_alloc_page(gfp_mask, order))
        return false;
    //CMA 内存分配标记处理
    *alloc_flags = gfp_to_alloc_flags_cma(gfp_mask, *alloc_flags);
    /* Dirty zone balancing only done in the fast path */
    ac->spread_dirty_pages = (gfp_mask & __GFP_WRITE);
    // 获取最高优先级的内存区域 zone
    // 后续内存分配则首先会在该内存区域中进行分配
    ac->preferred_zoneref = first_zones_zonelist(ac->zonelist,
                    ac->highest_zoneidx, ac->nodemask);
    return true;
}
```

prepare_alloc_pages 主要的任务就是在快速内存分配开始之前，做一些准备初始化的工作，其中最核心的就是从指定 NUMA 节点中，根据 gfp_mask 掩码中的内存区域修饰符获取可以进行内存分配的所有内存区域 zone （包括其他备用 NUMA 节点中包含的内存区域）。

struct pglist_data 结构中不仅包含了本 NUMA 节点中的所有内存区域，还包括了其他备用 NUMA 节点中的物理内存区域，当本节点中内存不足的情况下，内核会从备用 NUMA 节点中的内存区域进行跨节点内存分配。

```
typedef struct pglist_data {
    struct zone node_zones[MAX_NR_ZONES];
    struct zonelist node_zonelists[MAX_ZONELISTS];
    int nr_zones; /* number of populated zones in this node */
    ......
}pg_data_t;
```

可以根据 nid 和 gfp_mask 掩码中的物理内存区域描述符利用 node_zonelist 函数一次性获取允许进行内存分配的所有内存区域（所有 NUMA 节点）。

```
static inline struct zonelist *node_zonelist(int nid, gfp_t flags)
{
    return NODE_DATA(nid)->node_zonelists + gfp_zonelist(flags);
}
```

每个 NUMA 节点有两个 zonelist：

「全局 zonelist」：包含系统中所有有内存的 zone（优先本地节点，后跨节点），用于允许跨节点分配的场景；

「本地 zonelist」：仅包含当前节点自身的 zone，用于 __GFP_THISNODE 掩码（强制本地节点分配）的场景。

5.3.2.2 慢速分配入口函数 alloc_pages_slowpath 函数分析

慢速分配入口函数 alloc_pages_slowpath，包含了整个内存分配的核心流程。

```
static inline struct page *
__alloc_pages_slowpath(gfp_t gfp_mask, unsigned int order,
                       struct alloc_context *ac)
{
     ......... 初始化慢速内存分配路径下的相关参数 .......

restart:
      /*******配置变化后重试入口，确保重试时状态干净.******/
        ......当 cpuset/zonelist 配置变化时......
        ......从新补全分配标记......
        ......重新初始化迭代器......
        ......为新一轮分配尝试做准备......

        ......补全快速路径配置的分配参数......
```

```
      ......校验配置合法性......
      ......提前唤醒后台回收线程......

    /*******为分配做前置准备******/
    ......用 get_page_from_freelist（快速路径核心函数）......
    ......传入补全后的 alloc_flags 重试分配......
    ......若分配成功，直接跳转到 got_pg 标签返回页指针......
    ......跳过后续所有重量级操作......

    ......针对大阶页（costly_order）或非可移动页分配......
    ......优先触发“直接内存压缩”（整理碎片:......
    ......lloc_pages_direct_compact）......
    ......而非直接回收（降低分配成本）......

    retry:
    /*******通过“唤醒回收→调整标记→再次尝试→直接回收→再次压缩→重试校验”
            流程，多次尝试分配，是最核心的重量级操作模块。********/
    ......唤醒 kswapd, 调整 alloc_flags 使用更加激进的整内存分配策略......
    ......忽略 cpuset 限制,重置 nodemask 为 NULL 扩大分配范围......
    ......再次轻量尝试：调用 get_page_from_freelist 重试分配，成功则返回..
    ......直接内存回收,调用 __alloc_pages_direct_reclaim 同步回收内存,
           （当前进程阻塞），回收后重试分配......
    ......再次内存压缩：回收失败后，调用 __alloc_pages_direct_compact 再次
    ......压缩（调整优先级），整理碎片后重试分配......

    /******当回收、压缩均失败时，触发 OOM killer 杀死低优先级进程释放内存，
     ******是分配的最后兜底手段******/
    ......调用 __alloc_pages_may_oom 选择并杀死内存占用高、
    ......优先级低的进程，释放其物理页......
    ......若 OOM 后分配成功则返回    ......
    ......若 OOM 后分配失败，避免不检查水位阈值的内存分配陷入死循环.......
    ......若 OOM killer 有进展（杀死了进程）
    ......重置无进展次数并跳回 retry 重试......

    nopage:
    ......处理分配失败的收尾逻辑，检测配置变化并尝试重启......
    ......同时处理 __GFP_NOFAIL（强制不失败）标记的极端场景......
    ......警告非法场景......
    ......调用 __alloc_pages_cpuset_fallback ......
    ......访问最小预留内存（不耗尽系统预留）......
    ......若仍失败，调度其他进程等待内存释放......
    ......然后无限跳回 retry 重试（直到分配成功）......

  /*******分配失败时打印警告日志，成功则返回页指针，是函数的最终出口*****/
```

```
}
```

```
static inline struct page *
__alloc_pages_slowpath(gfp_t gfp_mask, unsigned int order,
                       struct alloc_context *ac)
{
    // 在慢速内存分配路径中可能会导致内核进行直接内存回收
    // 这里设置 __GFP_DIRECT_RECLAIM 表示允许内核进行直接内存回收
    bool can_direct_reclaim = gfp_mask & __GFP_DIRECT_RECLAIM;
    bool can_compact = gfp_compaction_allowed(gfp_mask);
     // 本次内存分配是否是针对大量内存页的分配，
    //内核定义 PAGE_ALLOC_COSTLY_ORDER = 3
    // 也就是说内存请求内存页的数量大于 2 ^ 3 = 8 个内存页时，
    //costly_order = true，后续会影响是否进行 OOM
    const bool costly_order = order > PAGE_ALLOC_COSTLY_ORDER;
    //用于指向成功申请的内存
    struct page *page = NULL;
    //内存分配标识，后续会根据不同标识进入到不同的内存分配逻辑处理分支
    unsigned int alloc_flags;
    //后续用于记录直接内存回收流程回收了多少内存页
    unsigned long did_some_progress;
    //关于内存整理相关参数
    enum compact_priority compact_priority;
    enum compact_result compact_result;
    int compaction_retries;
    int no_progress_loops;
    unsigned int cpuset_mems_cookie;
    unsigned int zonelist_iter_cookie;
    int reserve_flags;

restart:
    compaction_retries = 0;
    no_progress_loops = 0;
```

```
compact_priority = DEF_COMPACT_PRIORITY;
cpuset_mems_cookie = read_mems_allowed_begin();
zonelist_iter_cookie = zonelist_iter_begin();
// 在之前的快速内存分配路径下设置的相关分配策略比较保守，
//不是很激进，用于在 WMARK_LOW 水位线之上进行快速内存分配
// 走到这里表示快速内存分配失败，此时空闲内存严重不足了
// 所以在慢速内存分配路径下需要重新设置更加激进的内存分配策略，
//采用更大的代价来分配内存
alloc_flags = gfp_to_alloc_flags(gfp_mask, order);

// 重新按照新的配置设置内存区域优先级计算 zonelist
//的迭代起点（最高优先级的 zone）
// fast path 和 slow path 的设置不同，所以这里需要重新计算
ac->preferred_zoneref = first_zones_zonelist(ac->zonelist,
                ac->highest_zoneidx, ac->nodemask);
//如果没有合适的内存分配区域，则跳转到 nopage ，内存分配失败
if (!ac->preferred_zoneref->zone)
    goto nopage;
if (cpusets_insane_config() && (gfp_mask & __GFP_HARDWALL)) {
    struct zoneref *z = first_zones_zonelist(ac->zonelist,
                ac->highest_zoneidx,
                &cpuset_current_mems_allowed);
    if (!z->zone)
        goto nopage;
}
//唤醒所有的 kswapd 进程异步回收内存
if (alloc_flags & ALLOC_KSWAPD)
    wake_all_kswapds(order, gfp_mask, ac);
 // 此时所有的 kswapd 进程已经被唤醒，正在异步进行内存回收
// 之前我们已经在 gfp_to_alloc_flags 方法中重新调整了 alloc_flags
// 换成了一套更加激进的内存分配策略，注意此时是在 WMARK_MIN
//水位线之上进行内存分配
    // 调整后的 alloc_flags 很可能会立即成功，因此这里先尝试一下
page = get_page_from_freelist(gfp_mask, order, alloc_flags, ac);

// 内存分配成功，跳转到 got_pg 直接返回 page
if (page)
    goto got_pg;
// 对于分配大内存来说 costly_order = true (超过 8 个内存页)，
//需要首先进行内存整理，这样内核可以避免直接内存回收从而获取更多的
//连续空闲内存页
// 对于需要分配不可移动的高阶内存的情况，也需要先进行内存整理，
//防止永久内存碎片
if (can_direct_reclaim && can_compact &&
```

```
            (costly_order ||
               (order > 0 && ac->migratetype != MIGRATE_MOVABLE))
            && !gfp_pfmemalloc_allowed(gfp_mask)) {
        page = __alloc_pages_direct_compact(gfp_mask, order,
                        alloc_flags, ac,
                        INIT_COMPACT_PRIORITY,
                        &compact_result);// 进行直接内存整理，获取更多的连续
空闲内存防止内存碎片
        if (page)
            goto got_pg;
        if (costly_order && (gfp_mask & __GFP_NORETRY)) {
        // 流程走到这里表示经过内存整理之后依然没有足够的内存供分配
       // 但是设置了 NORETRY 标识不允许重试，那么就直接失败，跳转到 nopage
            if (compact_result == COMPACT_SKIPPED ||
                compact_result == COMPACT_DEFERRED)
                goto nopage;
         // 同步内存整理开销太大，后续开启异步内存整理
            compact_priority = INIT_COMPACT_PRIORITY;
        }
    }
retry:
    // 确保所有 kswapd 进程不要意外进入睡眠状态
    if (alloc_flags & ALLOC_KSWAPD)
        wake_all_kswapds(order, gfp_mask, ac);
    // 流程走到这里，说明在 WMARK_MIN 水位线之上也分配内存失败了
    // 并且经过内存整理之后，内存分配仍然失败，说明当前内存容量已经严重不足
    // 接下来就需要使用更加激进的非常手段来尝试内存分配（忽略掉内存水位线）
/  //继续修 alloc_flags 保存在 reserve_flags 中
    reserve_flags = __gfp_pfmemalloc_flags(gfp_mask);
    if (reserve_flags)
        alloc_flags = gfp_to_alloc_flags_cma(gfp_mask, reserve_flags) |
                      (alloc_flags & ALLOC_KSWAPD);

        // 如果内存分配可以任意跨节点分配（忽略内存分配策略）
        //这里需要置 nodemask 以及 zonelist。
    if (!(alloc_flags & ALLOC_CPUSET) || reserve_flags) {
        //这里的内存分配是高优先级系统级别的内存分配，不是面向用户的
        ac->nodemask = NULL;
        ac->preferred_zoneref = first_zones_zonelist(ac->zonelist,
                    ac->highest_zoneidx, ac->nodemask);
    }
    // 这里使用重新调整的 zonelist 和 alloc_flags 在尝试进行一次内存分配
    // 注意此次的内存分配是忽略内存水位线的 ALLOC_NO_WATERMARKS
    page = get_page_from_freelist(gfp_mask, order, alloc_flags, ac);
```

```
    if (page)
        goto got_pg;

    // 在忽略内存水位线的情况下仍然分配失败，现在内核就需要进行直接内存回收了
    if (!can_direct_reclaim)
        goto nopage; // 如果进程不允许进行直接内存回收，则只能分配失败

    //避免直接内存回收（direct reclaim）的递归调用
    if (current->flags & PF_MEMALLOC)
        goto nopage;

    // 开始直接内存回收
    page = __alloc_pages_direct_reclaim(gfp_mask, order, alloc_flags, ac,
                                        &did_some_progress);
    if (page)
        goto got_pg;
    /* Try direct compaction and then allocating */
    page = __alloc_pages_direct_compact(gfp_mask, order, alloc_flags, ac,
                                        compact_priority, &compact_result);
    if (page)
        goto got_pg;

    // 在内存直接回收和整理全部失败之后，如果不允许重试，则只能失败
    if (gfp_mask & __GFP_NORETRY)
        goto nopage;

    // 后续会触发 OOM 来释放更多的内存，
    //这里需要判断本次内存分配是否需要分配大量的内存页
    //（大于 8 ） costly_order = true
    //如果是的话则内核认为即使执行 OOM 也未必会满足这么多的内存页分配需求.
    //所以还是直接失败比较好，不再执行 OOM，除非设置 __GFP_RETRY_MAYFAIL
    if (costly_order && (!can_compact ||
                 !(gfp_mask & __GFP_RETRY_MAYFAIL)))
        goto nopage;

    // 流程走到这里说明我们已经尝试了所有措施内存依然分配失败了，
    //此时内存已经非常危急了。
    // 走到这里说明进程允许内核进行重试流程，
    //但在开始重试之前，内核需要判断是否应该进行重试,重试标准：
    //1 如果内核已经重试了 MAX_RECLAIM_RETRIES (16) 次仍然失败，
    //则放弃重试执行后续 OOM。
    //2 如果内核将所有可选内存区域中的所有可回收页面全部回收之后，
    //仍然无法满足内存的分配，那么放弃重试执行后续 OOM
    if (should_reclaim_retry(gfp_mask, order, ac, alloc_flags,
```

```
                did_some_progress > 0, &no_progress_loops))
        goto retry;
    //如果内核判断不应进行直接内存回收的重试，
    //这里还需要判断下是否应该进行内存整理的重试。
    //did_some_progress 表示上次直接内存回收，具体回收了多少内存页
    // 如果 did_some_progress = 0 则没有必要在进行内存整理重试了，
    //因为内存整理的实现依赖于足够的空闲内存量
    if (did_some_progress > 0 && can_compact &&
            should_compact_retry(ac, order, alloc_flags,
                compact_result, &compact_priority,
                &compaction_retries))
        goto retry;

       //检查 cpuset 配置 或者 zonelist 是否发生了更新
       //如果，已经被更新重新进入 restart 流程分配
    if (check_retry_cpuset(cpuset_mems_cookie, ac) ||
        check_retry_zonelist(zonelist_iter_cookie))
        goto restart;

    // 最后的杀手锏，进行 OOM，选择一个得分最高的进程，释放其占用的内存
    page = __alloc_pages_may_oom(gfp_mask, order, ac, &did_some_progress);
    if (page)
        goto got_pg;

    /* Avoid allocations with no watermarks from looping endlessly */
    if (tsk_is_oom_victim(current) &&
        (alloc_flags & ALLOC_OOM ||
         (gfp_mask & __GFP_NOMEMALLOC)))
        goto nopage;

    // 只要 oom 产生了作用并释放了内存 did_some_progress > 0
    //就不断的进行重试
    if (did_some_progress) {
        no_progress_loops = 0;
        goto retry;
    }
nopage:
    if (check_retry_cpuset(cpuset_mems_cookie, ac) ||
        check_retry_zonelist(zonelist_iter_cookie))
        goto restart;
    if (gfp_mask & __GFP_NOFAIL) {
        if (WARN_ON_ONCE_GFP(!can_direct_reclaim, gfp_mask))
            goto fail;
        WARN_ON_ONCE_GFP(current->flags & PF_MEMALLOC, gfp_mask);
```

```
        WARN_ON_ONCE_GFP(costly_order, gfp_mask);
        page = __alloc_pages_cpuset_fallback(gfp_mask, order,
ALLOC_MIN_RESERVE, ac);
        if (page)
            goto got_pg;
        cond_resched();
        goto retry;
    }
fail:
    warn_alloc(gfp_mask, ac->nodemask,
            "page allocation failure: order:%u", order);
got_pg:
    return page;
}
```

retry 分支包含的是更加激进的内存分配逻辑，所以在一开始需要调用 __gfp_pfmemalloc_flags 函数来重新调整内存分配策略，调整后的策略为：后续内存分配会忽略水位线的影响，并且允许内核从紧急预留内存中获取内存。

```
static inline int __gfp_pfmemalloc_flags(gfp_t gfp_mask)
{
    // 如果不允许从紧急预留内存中分配，则不改变 alloc_flags
    if (unlikely(gfp_mask & __GFP_NOMEMALLOC))
        return 0;

    // 如果允许从紧急预留内存中分配，则后面的内存分配会忽略内存水位线的限制
    if (gfp_mask & __GFP_MEMALLOC)
        return ALLOC_NO_WATERMARKS;

    // 当前进程处于软中断上下文并且进程设置了 PF_MEMALLOC 标识
    // 则忽略内存水位线
    if (in_serving_softirq() && (current->flags & PF_MEMALLOC))
        return ALLOC_NO_WATERMARKS;

    // 当前进程不在任何中断上下文中
    if (!in_interrupt()) {
        if (current->flags & PF_MEMALLOC)
            // 忽略内存水位线
            return ALLOC_NO_WATERMARKS;
        else if (oom_reserves_allowed(current))
            //当前进程允许进行 OOM
            return ALLOC_OOM;
    }
    return 0;
}
```

在调整好更加激进的内存分配策略 alloc_flags 之后，内核会调用 get_page_from_freelist 首先尝试从伙伴系统中进行一次内存分配，这时会有很大概率促使内存分配成功。此次尝试进行的内存分配会忽略内存水位线：ALLOC_NO_WATERMARKS。

如果在忽略内存水位线的情况下，内存依然分配失败，则调用 __alloc_pages_direct_reclaim 进行直接内存回收 direct_reclaim 。

经过 direct_reclaim 之后，仍然没有足够的内存可供分配的话，那么内核会调用 __alloc_pages_direct_compact 再次进行直接内存整理 direct_compact 。

如果 direct_compact 之后还是没有足够的内存，那么现在内核已经处于绝境了，是时候使用杀手锏：触发 OOM 机制杀死得分最高的进程以获取更多的空闲内存。

但是在进行 OOM 之前，内核还是需要经过一系列的判断，如果 costly_order = true，表示此次内存分配的内存页大于 8 个页，内核会认为这是一次代价比较大的分配行为，况且此时内存已经非常危急，严重不足。在这种情况下内核认为即使触发了 OOM，也无法获取这么多的内存，依然无法满足内存分配。所以当 costly_order = true 时，内核不会触发 OOM，直接跳转到 nopage 分支，除非设置了 __GFP_RETRY_MAYFAIL 内存分配策略或者支持内存压缩（compact）：

```
    if (costly_order && (!can_compact ||
                !(gfp_mask & __GFP_RETRY_MAYFAIL)))
        goto nopage;
```

下面内核也不会直接开始 OOM，而是进入到重试流程，在重试流程开始之前内核需要调用 should_reclaim_retry 判断是否应该进行重试，重试标准：

1. 如果内核已经重试了 MAX_RECLAIM_RETRIES (16) 次仍然失败，则放弃重试执行后续 OOM。
2. 如果内核将所有可选内存区域中的所有可回收页面全部回收之后，仍然无法满足内存的分配，那么放弃重试执行后续 OOM。

```
    if (should_reclaim_retry(gfp_mask, order, ac, alloc_flags,
                did_some_progress > 0, &no_progress_loops))
        goto retry;
```

如果 should_reclaim_retry = false，后面会进一步判断是否应该进行 direct_compact 的重试。

```
    if (did_some_progress > 0 && can_compact &&
            should_compact_retry(ac, order, alloc_flags,
                compact_result, &compact_priority,
                &compaction_retries))
        goto retry;
```

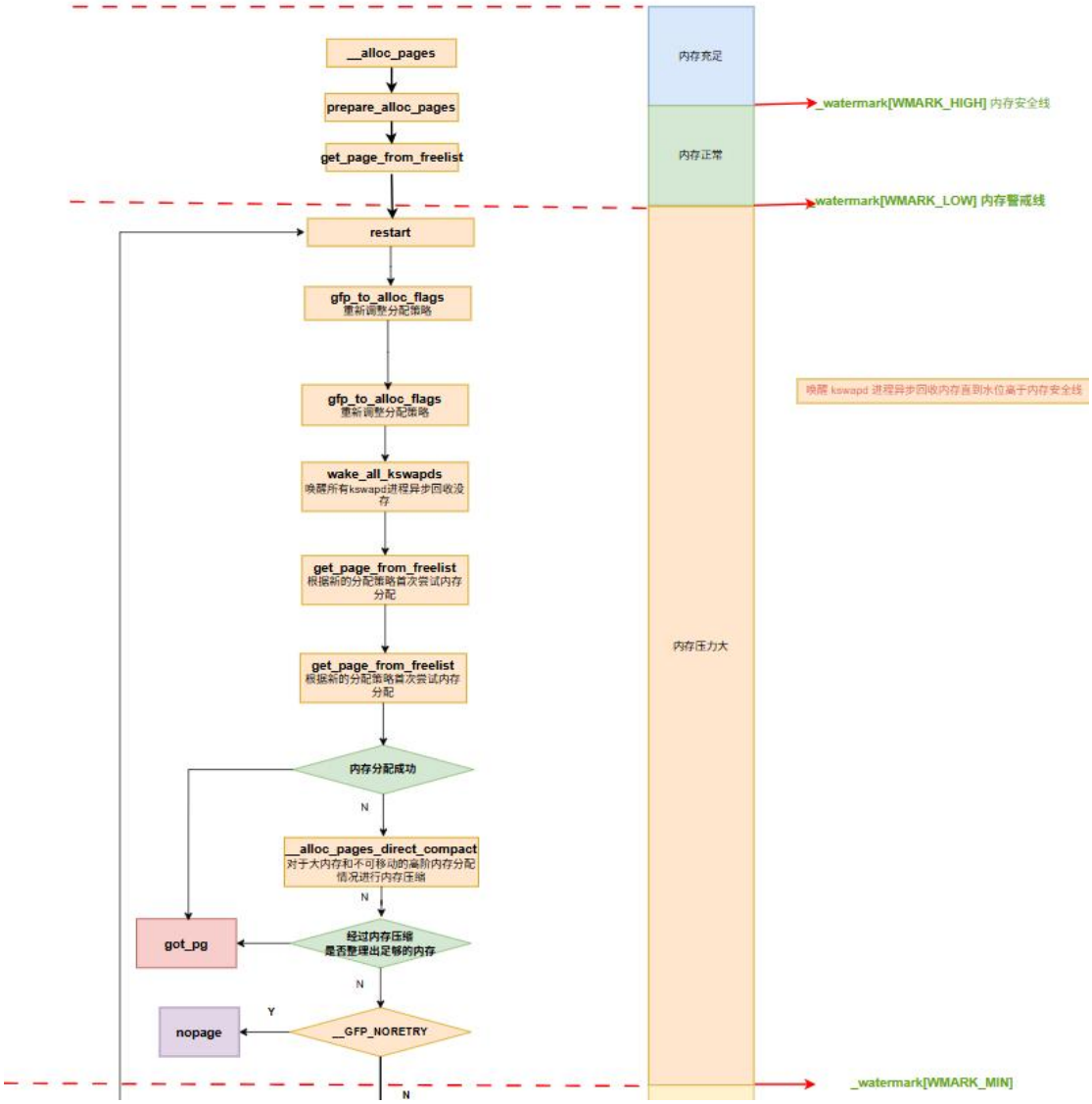
did_some_progress 表示上次直接内存回收具体回收了多少内存页，如果 did_some_progress = 0 则没有必要在进行内存整理重试了，因为内存整理的实现依赖于足够的空闲内存量。

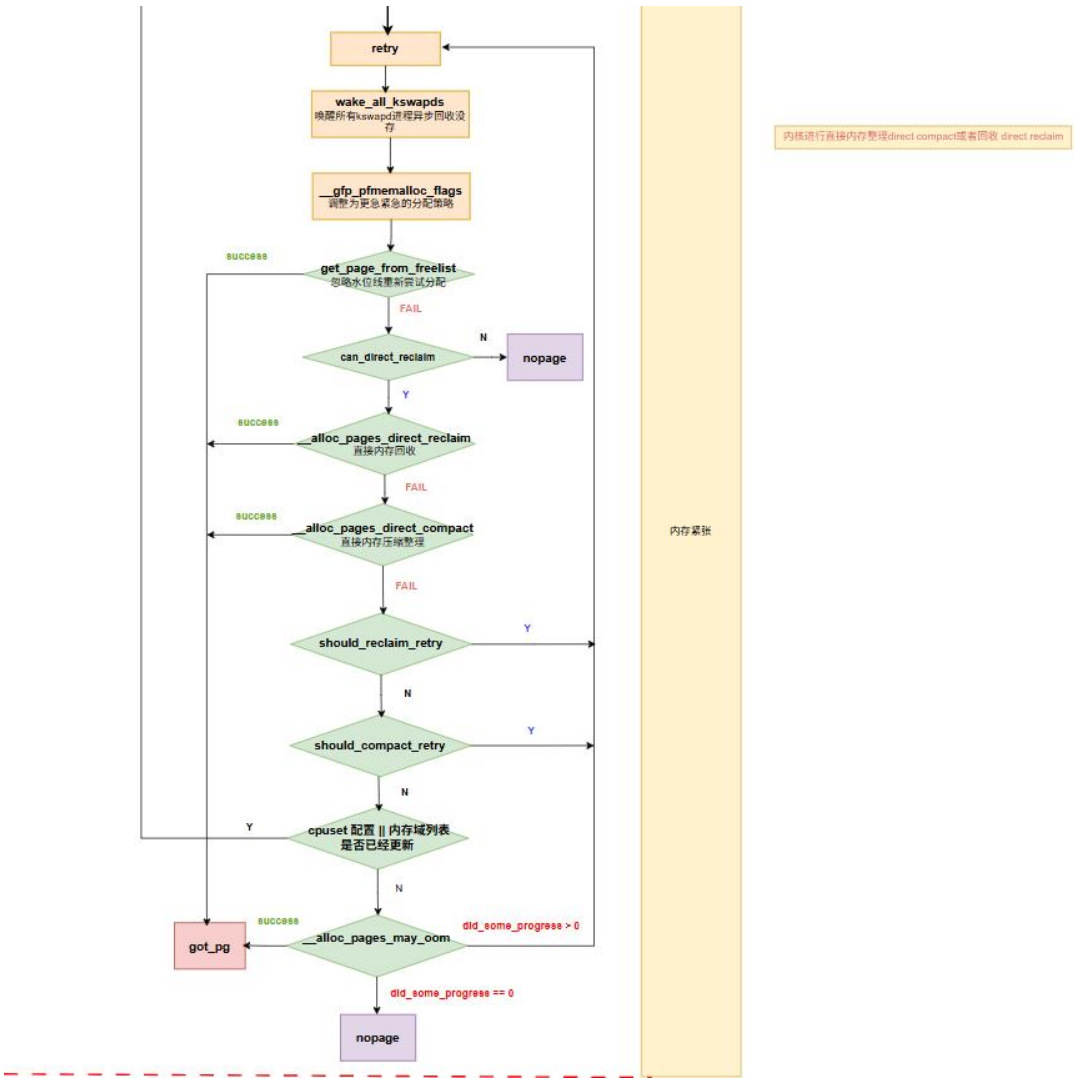
当这些所有的重试请求都被拒绝时，杀手锏 OOM 就开始登场了：

```
    page = __alloc_pages_may_oom(gfp_mask, order, ac, &did_some_progress);
    if (page)
        goto got_pg;
```

如果 OOM 之后并没有释放内存，那么就来到 nopage 分支处理。但是如果

did_some_progress > 0 表示 OOM 产生了作用，至少释放了一些内存那么就再次进行重试。





5.3.2.3 慢速分配函数 alloc_pages_slowpath 函数中 nopage 模块分析

函数走到 nopage 模块，表明内核已经尝试了包括 OOM 在内的所有回收内存的措施，但是仍然没有足够的内存来满足分配要求，看上去此次内存分配就要宣告失败了。

但是，如果内存分配策略中配置了 __GFP_NOFAIL，则表示此次内存分配非常的重要，不允许失败。内核会在这里不停的重试直到分配成功为止。

在 UMA 内存架构中：当 CPU 自己所在的本地 NUMA 节点内存不足时，CPU 就需要跨 NUMA 节点去访问其他内存节点，这种跨 NUMA 节点分配内存的行为就发生在这里，这种情况下 CPU 访问内存就会慢很多。

```
static inline struct page *
__alloc_pages_slowpath(gfp_t gfp_mask, unsigned int order,
                       struct alloc_context *ac)
{
      ......省略......
```

```
nopage:
    // 流程走到这里表明内核已经尝试了包括 OOM 在内的所有回收内存的动作。
    // 但是这些措施依然无法满足内存分配的需求，看上去内存分配到这里就应该失败了
    //检查 cpuset 配置 或者 zonelist 是否发生了更新
    //如果，已经被更新重新进入 restart 流程分配
    if (check_retry_cpuset(cpuset_mems_cookie, ac) ||
        check_retry_zonelist(zonelist_iter_cookie))
        goto restart;

    // 但是如果设置了 __GFP_NOFAIL 表示不允许内存分配失败，
    //那么接下来就会进入分支进行处理
    if (gfp_mask & __GFP_NOFAIL) {
    //如果不允许进行直接内存回收，则跳转至 fail 分支宣告失败
        if (WARN_ON_ONCE_GFP(!can_direct_reclaim, gfp_mask))
            goto fail;
    // 此时内核已经无法通过回收内存来获取可供分配的空闲内存了
    // 对于 PF_MEMALLOC 类型的内存分配请求，内核现在无能为力，
    //只能不停的进行 retry 重试。
        WARN_ON_ONCE_GFP(current->flags & PF_MEMALLOC, gfp_mask);
     // 对于需要分配 8 个内存页以上的大内存分配，
    //并且设置了不可失败标识 __GFP_NOFAIL
    // 内核现在也无能为力，毕竟现实是已经没有空闲内存了，
    //只是给出一些告警信息
        WARN_ON_ONCE_GFP(costly_order, gfp_mask);
    //在 __GFP_NOFAIL 情况下，尝试进行跨 NUMA 节点内存分配
        page = __alloc_pages_cpuset_fallback(gfp_mask, order,
ALLOC_MIN_RESERVE, ac);
        if (page)
            goto got_pg;
     // 在进行内存分配重试流程之前，需要让 CPU 重新调度到其他进程上
    // 运行一会其他进程，因为毕竟此时内存已经严重不足
    // 立马重试的话只能浪费过多时间在搜索空闲内存上，导致其他进程处于饥饿状态。
        cond_resched();
    //跳转到 retry 分支，重试内存分配流程
        goto retry;
    }

 ......省略......
}
```

在 nopage 分支中决定开始重试之前，内核不能立即进行重试流程，因为之前已经经历过那么多严格激进的内存回收策略仍然没有足够的内存，内存现状非常紧急。所以我们有理由相信，如果内核立即开始重试的话，依然没有什么效果，反而会浪费过多时间在搜索空闲内存上，导致其他进程处于饥饿状态。

所以在开始重试之前，内核会调用 cond_resched() 让 CPU 重新调度到其他进程上，让

其他进程也运行一会，与此同时 kswapd 进程一直在后台异步回收着内存。当 CPU 重新调度回当前进程时，说不定 kswapd 进程已经回收了足够多的内存，重试成功的概率会大大增加同时又避免了资源的无谓消耗。

内存分配流程中涉及到的三个重要辅助函数：prepare_alloc_pages，__alloc_pages_slowpath，get_page_from_freelist 。prepare_alloc_pages，__alloc_pages_slowpath 函数主要是根据不同的空闲内存剩余容量调整内存的分配策略，尽量使内存分配行为尽最大可能成功。而内核中执行具体内存分配动作是 get_page_from_freelist 函数。它和内核中的伙伴系统有着千丝万缕的联系。